

Untitled-1

July 24, 2025

OUTLIERS, NULL VALUES AND CORRELATION

LOAD LIBRARIES

```
[112]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

LOAD DATA

```
[113]: df = pd.read_csv('/home/kimath/Downloads/sSTUDENT/
↳Expanded_data_with_more_features.csv')
```

df

```
[113]:
```

	Unnamed: 0	Gender	EthnicGroup	ParentEduc	LunchType	\
0	0	female	NaN	bachelor's degree	standard	
1	1	female	group C	some college	standard	
2	2	female	group B	master's degree	standard	
3	3	male	group A	associate's degree	free/reduced	
4	4	male	group C	some college	standard	
...	...	...	...	...	...	
30636	816	female	group D	high school	standard	
30637	890	male	group E	high school	standard	
30638	911	female	NaN	high school	free/reduced	
30639	934	female	group D	associate's degree	standard	
30640	960	male	group B	some college	standard	

	TestPrep	ParentMaritalStatus	PracticeSport	IsFirstChild	NrSiblings	\
0	none	married	regularly	yes	3.0	
1	NaN	married	sometimes	yes	0.0	
2	none	single	sometimes	yes	4.0	
3	none	married	never	no	1.0	
4	none	married	sometimes	yes	0.0	
...	...	...	...	...	...	
30636	none	single	sometimes	no	2.0	
30637	none	single	regularly	no	1.0	
30638	completed	married	sometimes	no	1.0	
30639	completed	married	regularly	no	3.0	

30640	none	married	never	no	1.0
-------	------	---------	-------	----	-----

	TransportMeans	WklyStudyHours	MathScore	ReadingScore	WritingScore
0	school_bus	< 5	71	71	74
1	NaN	5 - 10	69	90	88
2	school_bus	< 5	87	93	91
3	NaN	5 - 10	45	56	42
4	school_bus	5 - 10	76	78	75
...	...	...	...	...	...
30636	school_bus	5 - 10	59	61	65
30637	private	5 - 10	58	53	51
30638	private	5 - 10	61	70	67
30639	school_bus	5 - 10	82	90	93
30640	school_bus	5 - 10	64	60	58

[30641 rows x 15 columns]

UNDERSTAND THE DATA

```
[114]: print(df.head())
```

	Unnamed: 0	Gender	EthnicGroup	ParentEduc	LunchType	TestPrep	\
0	0	female	NaN	bachelor's degree	standard	none	
1	1	female	group C	some college	standard	NaN	
2	2	female	group B	master's degree	standard	none	
3	3	male	group A	associate's degree	free/reduced	none	
4	4	male	group C	some college	standard	none	

	ParentMaritalStatus	PracticeSport	IsFirstChild	NrSiblings	TransportMeans	\
0	married	regularly	yes	3.0	school_bus	
1	married	sometimes	yes	0.0	NaN	
2	single	sometimes	yes	4.0	school_bus	
3	married	never	no	1.0	NaN	
4	married	sometimes	yes	0.0	school_bus	

	WklyStudyHours	MathScore	ReadingScore	WritingScore
0	< 5	71	71	74
1	5 - 10	69	90	88
2	< 5	87	93	91
3	5 - 10	45	56	42
4	5 - 10	76	78	75

```
[115]: df.tail()
```

```
[115]:
```

	Unnamed: 0	Gender	EthnicGroup	ParentEduc	LunchType	\
30636	816	female	group D	high school	standard	
30637	890	male	group E	high school	standard	
30638	911	female	NaN	high school	free/reduced	

30639	934	female	group D	associate's degree	standard
30640	960	male	group B	some college	standard

	TestPrep	ParentMaritalStatus	PracticeSport	IsFirstChild	NrSiblings	\
30636	none	single	sometimes	no	2.0	
30637	none	single	regularly	no	1.0	
30638	completed	married	sometimes	no	1.0	
30639	completed	married	regularly	no	3.0	
30640	none	married	never	no	1.0	

	TransportMeans	WklyStudyHours	MathScore	ReadingScore	WritingScore
30636	school_bus	5 - 10	59	61	65
30637	private	5 - 10	58	53	51
30638	private	5 - 10	61	70	67
30639	school_bus	5 - 10	82	90	93
30640	school_bus	5 - 10	64	60	58

```
[116]: print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30641 entries, 0 to 30640
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            30641 non-null  int64
1   Gender                30641 non-null  object
2   EthnicGroup           28801 non-null  object
3   ParentEduc            28796 non-null  object
4   LunchType             30641 non-null  object
5   TestPrep              28811 non-null  object
6   ParentMaritalStatus   29451 non-null  object
7   PracticeSport         30010 non-null  object
8   IsFirstChild          29737 non-null  object
9   NrSiblings            29069 non-null  float64
10  TransportMeans        27507 non-null  object
11  WklyStudyHours        29686 non-null  object
12  MathScore             30641 non-null  int64
13  ReadingScore          30641 non-null  int64
14  WritingScore          30641 non-null  int64
dtypes: float64(1), int64(4), object(10)
memory usage: 3.5+ MB
None
```

```
[117]: df.describe()
```

```
[117]:
```

	Unnamed: 0	NrSiblings	MathScore	ReadingScore	WritingScore
count	30641.000000	29069.000000	30641.000000	30641.000000	30641.000000

mean	499.556607	2.145894	66.558402	69.377533	68.418622
std	288.747894	1.458242	15.361616	14.758952	15.443525
min	0.000000	0.000000	0.000000	10.000000	4.000000
25%	249.000000	1.000000	56.000000	59.000000	58.000000
50%	500.000000	2.000000	67.000000	70.000000	69.000000
75%	750.000000	3.000000	78.000000	80.000000	79.000000
max	999.000000	7.000000	100.000000	100.000000	100.000000

CLEAN THE DATA

Key factors that guide whether to DROP or FILL

1. How much data is missing?

Small % of NULLs (e.g. < 5%)

Usually better to fill (imputation is easy and preserves data).

Large % of NULLs (e.g. > 50%)

Often better to drop the whole column - it doesn't carry reliable info anymore.

2. Importance of the feature

Is the column critical for your analysis or model?

If yes, do your best to fill it (using mean, median, mode, or more advanced prediction).

If not important, you can drop it to simplify your dataset.

Example: If you're predicting student performance, dropping math score would be bad — you must find a way to fill it.

3. Type of data

Numerical data

Fill with mean (if normal) or median (if skewed/outliers).

Categorical data

Fill with mode (most common value).

If the NULLs appear in rows where other data is missing too → maybe drop the whole row.

4. Size of the dataset

Big dataset: Dropping a few rows usually doesn't hurt.

Small dataset: Dropping rows may lose too much information → better to fill.

5. Impact on results

Dropping too much can cause bias → your data no longer represents reality.

Filling poorly can distort relationships → so always explain how you filled.

```
[118]: print(df.isnull().sum())
```

```

Unnamed: 0          0
Gender              0
EthnicGroup        1840
ParentEduc         1845
LunchType          0
TestPrep           1830
ParentMaritalStatus 1190
PracticeSport       631
IsFirstChild        904
NrSiblings          1572
TransportMeans      3134
WklyStudyHours      955
MathScore           0
ReadingScore        0
WritingScore        0
dtype: int64

```

```
[119]: print(round(df.isnull().sum() / len(df) * 100, 2))
```

```

Unnamed: 0          0.00
Gender              0.00
EthnicGroup         6.01
ParentEduc          6.02
LunchType           0.00
TestPrep            5.97
ParentMaritalStatus 3.88
PracticeSport        2.06
IsFirstChild         2.95
NrSiblings           5.13
TransportMeans       10.23
WklyStudyHours        3.12
MathScore            0.00
ReadingScore         0.00
WritingScore         0.00
dtype: float64

```

```
[120]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30641 entries, 0 to 30640
Data columns (total 15 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Unnamed: 0          30641 non-null  int64
1   Gender              30641 non-null  object
2   EthnicGroup         28801 non-null  object
3   ParentEduc          28796 non-null  object
4   LunchType           30641 non-null  object

```

```

5   TestPrep                28811 non-null object
6   ParentMaritalStatus     29451 non-null object
7   PracticeSport           30010 non-null object
8   IsFirstChild            29737 non-null object
9   NrSiblings              29069 non-null float64
10  TransportMeans          27507 non-null object
11  WklyStudyHours          29686 non-null object
12  MathScore               30641 non-null int64
13  ReadingScore            30641 non-null int64
14  WritingScore            30641 non-null int64
dtypes: float64(1), int64(4), object(10)
memory usage: 3.5+ MB

```

OPTION 1: FILL THE NULL VALUES

When to fill with the MEAN

Use mean when:

The numerical data is normally distributed.

There are no extreme outliers that could distort the average.

Why? The mean represents the central tendency accurately when data is balanced.

When to fill with the MEDIAN

Use median when:

The data is skewed (not symmetric) - has a long tail left or right.

There are outliers that pull the mean away from the typical value.

Why? The median is robust — it ignores extreme high or low values, so it shows the true “middle” of your data.

```

[121]: #numerical_cols = ['MathScore', 'ReadingScore', 'WritingScore', 'NrSiblings']
       #for col in numerical_cols:
       #    df[col] = df[col].fillna(df[col].median())

```

```

[122]: #categorical_cols = ['ParentEduc', 'EthnicGroup', 'TestPrep',
       ↪ 'ParentMaritalStatus', 'PracticeSport', 'IsFirstChild', 'TestPrep',
       ↪ 'TransportMeans', 'WklyStudyHours']
       #for col in categorical_cols:
       #    df[col] = df[col].fillna(df[col].mode()[0])

```

```

[123]: #print(df.isnull().sum())

```

OPTION 2: DROPPING THE NULL VALUES

```

[124]: df_cleaned = df.dropna()

```

```
[125]: df_cleaned.isnull().sum()
```

```
[125]: Unnamed: 0      0
      Gender      0
      EthnicGroup  0
      ParentEduc  0
      LunchType   0
      TestPrep    0
      ParentMaritalStatus  0
      PracticeSport  0
      IsFirstChild  0
      NrSiblings   0
      TransportMeans  0
      WklyStudyHours  0
      MathScore    0
      ReadingScore  0
      WritingScore  0
      dtype: int64
```

```
[126]: df_cleaned.shape
```

```
[126]: (19243, 15)
```

```
[127]: df_cleaned.dtypes
```

```
[127]: Unnamed: 0      int64
      Gender      object
      EthnicGroup  object
      ParentEduc  object
      LunchType   object
      TestPrep    object
      ParentMaritalStatus  object
      PracticeSport  object
      IsFirstChild  object
      NrSiblings   float64
      TransportMeans  object
      WklyStudyHours  object
      MathScore    int64
      ReadingScore  int64
      WritingScore  int64
      dtype: object
```

```
[128]: df_cleaned['NrSiblings']=df_cleaned['NrSiblings'].astype(int)
```

```
/tmp/ipykernel_5880/1943537215.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_cleaned['NrSiblings']=df_cleaned['NrSiblings'].astype(int)
```

```
[129]: df_cleaned.dtypes
```

```
[129]: Unnamed: 0          int64
      Gender          object
      EthnicGroup      object
      ParentEduc       object
      LunchType        object
      TestPrep         object
      ParentMaritalStatus object
      PracticeSport     object
      IsFirstChild      object
      NrSiblings       int64
      TransportMeans    object
      WklyStudyHours    object
      MathScore         int64
      ReadingScore      int64
      WritingScore      int64
      dtype: object
```

```
[130]: df_cleaned.shape
```

```
[130]: (19243, 15)
```

```
[131]: df_cleaned.drop_duplicates()
```

```
[131]:
```

	Unnamed: 0	Gender	EthnicGroup	ParentEduc	LunchType	\
2	2	female	group B	master's degree	standard	
4	4	male	group C	some college	standard	
5	5	female	group B	associate's degree	standard	
6	6	female	group B	some college	standard	
7	7	male	group B	some college	free/reduced	
...	...	...	...	...	...	
30635	794	male	group C	some college	standard	
30636	816	female	group D	high school	standard	
30637	890	male	group E	high school	standard	
30639	934	female	group D	associate's degree	standard	
30640	960	male	group B	some college	standard	

	TestPrep	ParentMaritalStatus	PracticeSport	IsFirstChild	NrSiblings	\
2	none	single	sometimes	yes	4	
4	none	married	sometimes	yes	0	
5	none	married	regularly	yes	1	

6	completed	widowed	never	no	1
7	none	married	sometimes	yes	1
...	...	...	...	...	...
30635	none	married	regularly	no	2
30636	none	single	sometimes	no	2
30637	none	single	regularly	no	1
30639	completed	married	regularly	no	3
30640	none	married	never	no	1

	TransportMeans	WklyStudyHours	MathScore	ReadingScore	WritingScore
2	school_bus	< 5	87	93	91
4	school_bus	5 - 10	76	78	75
5	school_bus	5 - 10	73	84	79
6	private	5 - 10	85	93	89
7	private	> 10	41	43	39
...	...	...	...	...	...
30635	school_bus	5 - 10	58	53	49
30636	school_bus	5 - 10	59	61	65
30637	private	5 - 10	58	53	51
30639	school_bus	5 - 10	82	90	93
30640	school_bus	5 - 10	64	60	58

[19243 rows x 15 columns]

```
[132]: df_cleaned.shape
```

```
[132]: (19243, 15)
```

OUTLIERS

When it's OK (and smart) to leave outliers 1. When the outliers are real and meaningful

If the unusual values are valid observations, they may contain important insights.

Example: A student who scored 100% when most score 60-80% → maybe they really are outstanding.

Example: A patient's medical bill is 10× higher → maybe they had a rare, costly procedure.

If you remove them, you hide important variation.

2. When your goal is to understand or predict rare events

Fraud detection → fraud transactions are the outliers.

Defect detection → the rare failing parts are the outliers.

High-risk insurance → the huge bills are the outliers you need to model.

3. When your model or summary can handle them

If you use robust statistics (like median, quantiles) → outliers won't skew results much.

If you use tree-based models (like Random Forest) → they handle outliers much better than linear regression.

4. When you can explain them in your report

Sometimes you keep them but note:

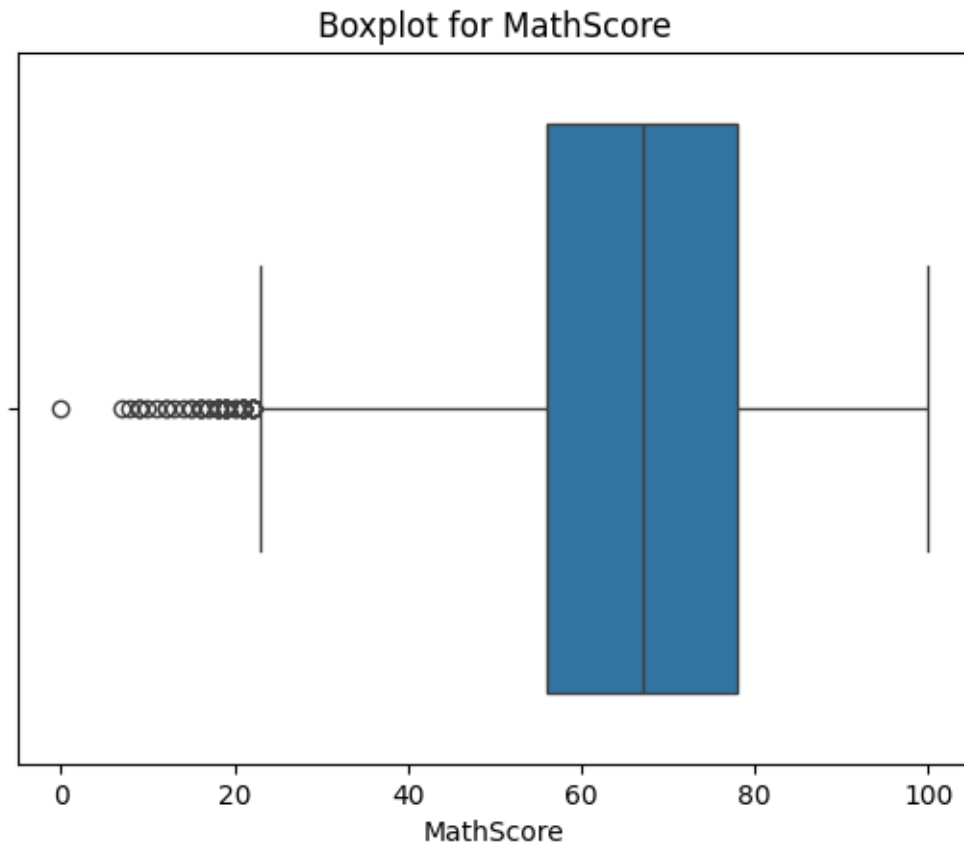
"Yes, this student's score is unusually high - it's valid."

"Yes, this charge is an extreme case - it's real."

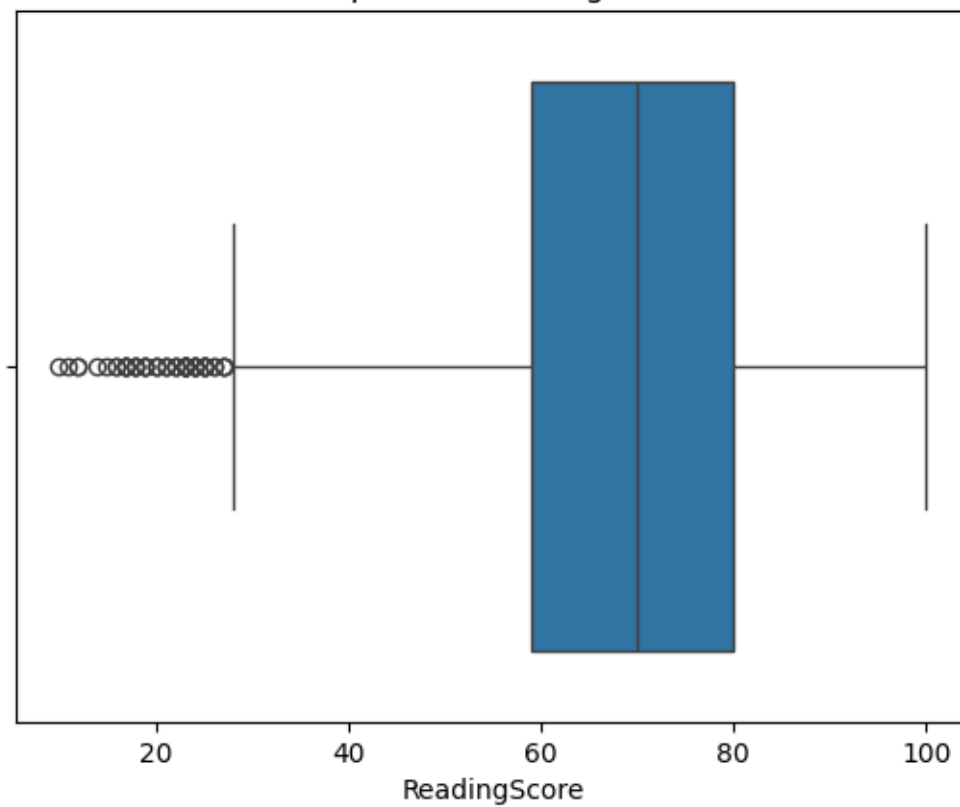
IDENTIFYING THE OUTLIERS USING BOXPLOT

```
[133]: numerical_cols = ['MathScore', 'ReadingScore', 'WritingScore', 'NrSiblings']

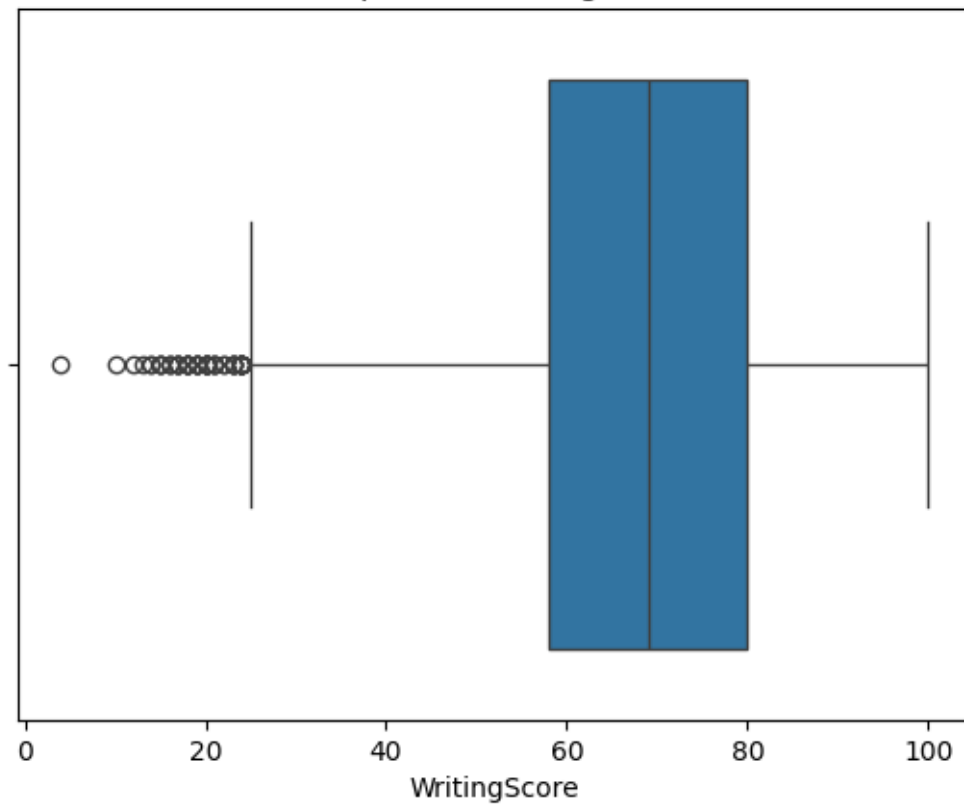
# 4 Plot boxplots
for col in numerical_cols:
    sns.boxplot(x=df_cleaned[col])
    plt.title(f'Boxplot for {col}')
    plt.show()
```

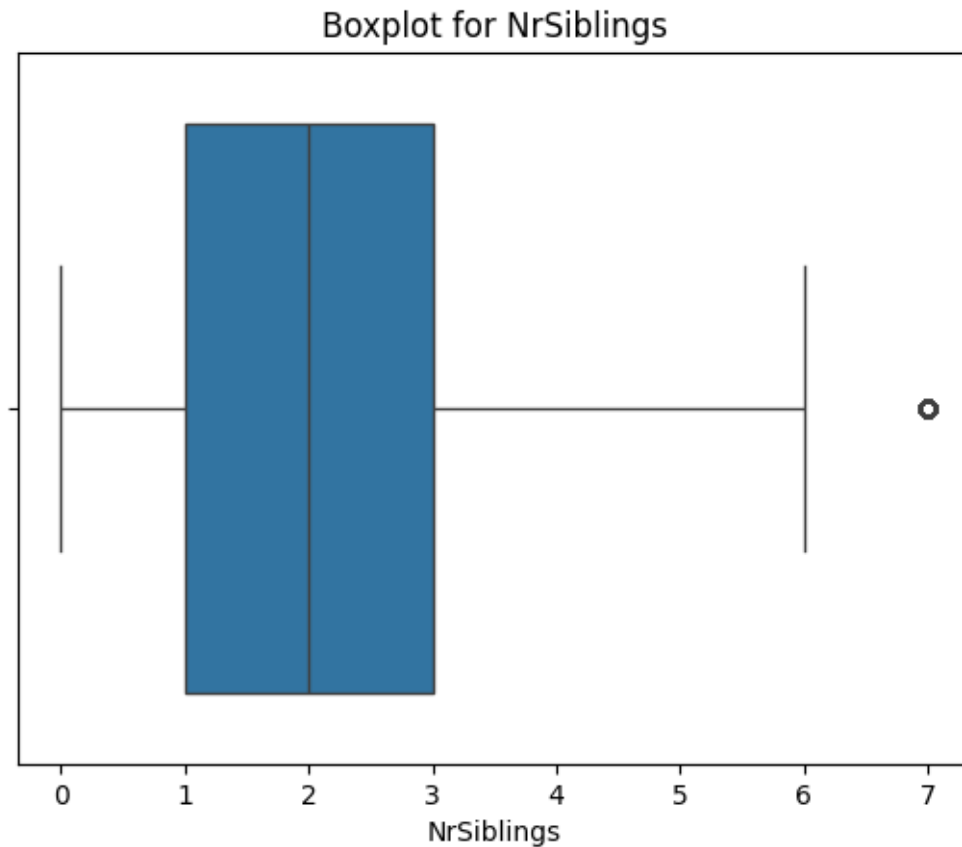


Boxplot for ReadingScore



Boxplot for WritingScore





IDENTIFYING THE OUTLIERS USING IQR

```
[134]: for col in numerical_cols:
        Q1 = df_cleaned[col].quantile(0.25)
        Q3 = df_cleaned[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        print(f"\nColumn: {col}")
        print(f"Q1: {Q1}")
        print(f"Q3: {Q3}")
        print(f"IQR: {IQR}")
        print(f"Lower bound: {lower_bound}")
        print(f"Upper bound: {upper_bound}")

        outliers = df_cleaned[(df_cleaned[col] < lower_bound) | (df_cleaned[col] >
↪upper_bound)]
        print(f"Number of outliers in {col}: {outliers.shape[0]}")
```

Column: MathScore
Q1: 56.0
Q3: 78.0
IQR: 22.0
Lower bound: 23.0
Upper bound: 111.0
Number of outliers in MathScore: 51

Column: ReadingScore
Q1: 59.0
Q3: 80.0
IQR: 21.0
Lower bound: 27.5
Upper bound: 111.5
Number of outliers in ReadingScore: 58

Column: WritingScore
Q1: 58.0
Q3: 80.0
IQR: 22.0
Lower bound: 25.0
Upper bound: 113.0
Number of outliers in WritingScore: 61

Column: NrSiblings
Q1: 1.0
Q3: 3.0
IQR: 2.0
Lower bound: -2.0
Upper bound: 6.0
Number of outliers in NrSiblings: 190

```
[135]: for col in numerical_cols:
        Q1 = df_cleaned[col].quantile(0.25)
        Q3 = df_cleaned[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        data = df_cleaned[(df_cleaned[col] >= lower_bound) & (df_cleaned[col] <=
↪upper_bound)]

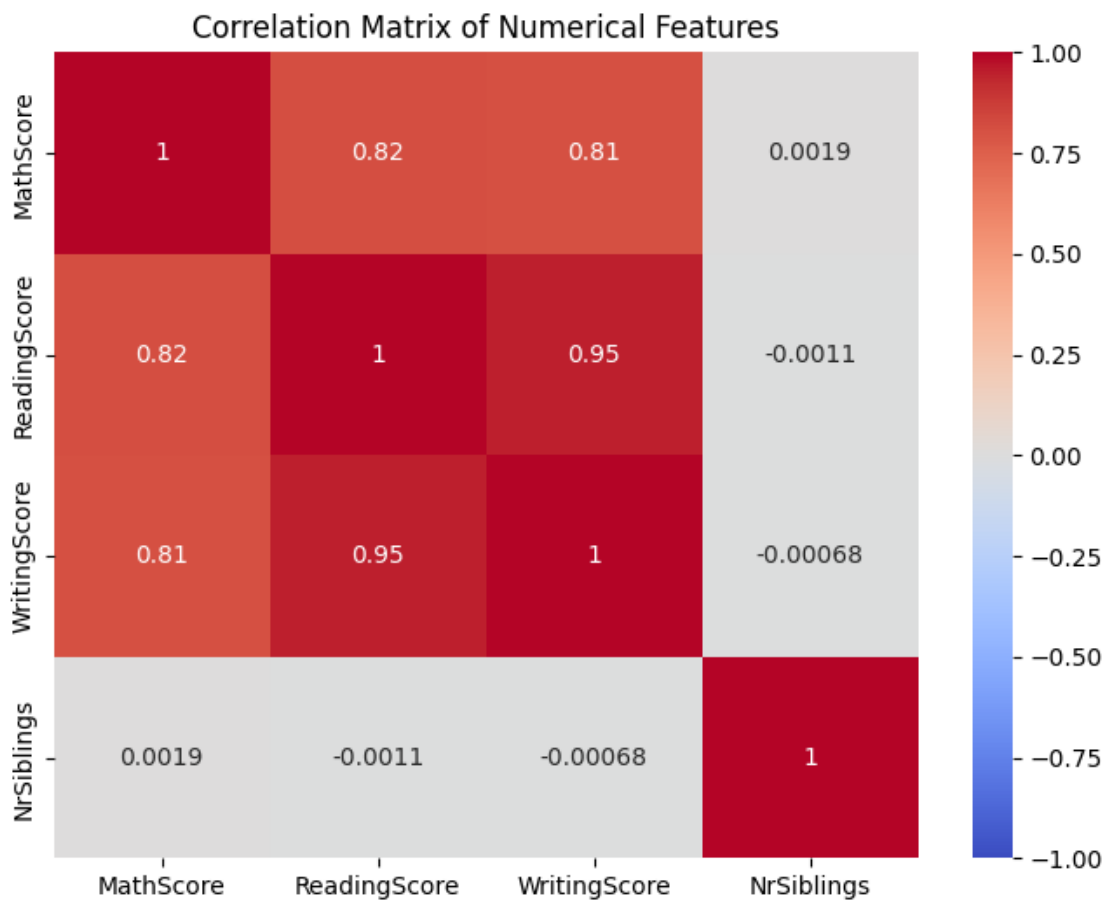
        print("Dataset shape after removing outliers:", data.shape)
```

Dataset shape after removing outliers: (19053, 15)

CORRELATION MATRIX

```
[136]: corr_matrix = data[numerical_cols].corr(method='pearson')

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1, center=0)
plt.title("Correlation Matrix of Numerical Features")
plt.show()
```



A correlation matrix shows the Pearson correlation coefficient between pairs of variables, ranging from -1 to 1:

1: Perfect positive correlation (as one variable increases, the other increases proportionally).
0: No correlation (no linear relationship).
-1: Perfect negative correlation (as one variable increases, the other decreases proportionally).
Values between 0 and 1 or 0 and -1 indicate the strength and direction of the linear relationship.

Red represents positive correlations (closer to 1).

Blue represents negative correlations (closer to -1).
 Gray/white indicates weak or no correlation (closer to 0).

The numbers in each cell are the correlation coefficients, annotated for clarity.

Diagonal Elements (1.0)

MathScore vs. MathScore, ReadingScore vs. ReadingScore, WritingScore vs. WritingScore, NrSiblings vs. NrSiblings
 Explanation: These are correlations of a variable with itself, always 1, indicating perfect correlation.

Off-Diagonal Elements

MathScore vs. ReadingScore (0.82):

Value: 0.82 (strong positive correlation).

Explanation: There's a strong linear relationship between math and reading scores. Students who score high in math also tend to score high in reading.

MathScore vs. WritingScore (0.81):

Value: 0.81 (strong positive correlation).

Explanation: Similar to the math-reading relationship, math and writing scores are strongly correlated.

ReadingScore vs. WritingScore (0.95):

Value: 0.95 (very strong positive correlation).

Explanation: This is the strongest correlation in the matrix, suggesting that reading and writing scores are highly related.

MathScore vs. NrSiblings (0.0037):

Value: 0.0037 (negligible correlation).

Explanation: There's virtually no linear relationship between math scores and the number of siblings.

ReadingScore vs. NrSiblings (0.0023):

Value: 0.0023 (negligible correlation).

Explanation: Similar to math, reading scores show no significant correlation with the number of siblings.

WritingScore vs. NrSiblings (0.0031):

Value: 0.0031 (negligible correlation).

Explanation: Again, writing scores are uncorrelated with the number of siblings, consistent with the other score variables.

Cross-Correlations Among NrSiblings and Scores:

Values: All near 0 (0.0023-0.0037).

Explanation: The near-zero correlations between NrSiblings and the score variables (MathScore, ReadingScore, WritingScore) indicate that the number of siblings does not significantly influence these scores.

SAVE THE CLEANED DATA

```
[137]: data.to_csv('AFTER_CLEANING.csv',index=False)
```

```
[138]: df1=pd.read_csv('/home/kimath/AFTER_CLEANING.csv')
df1
```

```
[138]:
```

	Unnamed: 0	Gender	EthnicGroup	ParentEduc	LunchType	\
0	2	female	group B	master's degree	standard	
1	4	male	group C	some college	standard	
2	5	female	group B	associate's degree	standard	
3	6	female	group B	some college	standard	
4	7	male	group B	some college	free/reduced	
...	...	...	...	...	...	
19048	794	male	group C	some college	standard	

19049	816	female	group D	high school	standard
19050	890	male	group E	high school	standard
19051	934	female	group D	associate's degree	standard
19052	960	male	group B	some college	standard

	TestPrep	ParentMaritalStatus	PracticeSport	IsFirstChild	NrSiblings	\
0	none	single	sometimes	yes	4	
1	none	married	sometimes	yes	0	
2	none	married	regularly	yes	1	
3	completed	widowed	never	no	1	
4	none	married	sometimes	yes	1	
...	...	...	...	...	...	
19048	none	married	regularly	no	2	
19049	none	single	sometimes	no	2	
19050	none	single	regularly	no	1	
19051	completed	married	regularly	no	3	
19052	none	married	never	no	1	

	TransportMeans	WklyStudyHours	MathScore	ReadingScore	WritingScore
0	school_bus	< 5	87	93	91
1	school_bus	5 - 10	76	78	75
2	school_bus	5 - 10	73	84	79
3	private	5 - 10	85	93	89
4	private	> 10	41	43	39
...	...	...	...	...	...
19048	school_bus	5 - 10	58	53	49
19049	school_bus	5 - 10	59	61	65
19050	private	5 - 10	58	53	51
19051	school_bus	5 - 10	82	90	93
19052	school_bus	5 - 10	64	60	58

[19053 rows x 15 columns]